Pre-Flight Drone Inspection Trainer

Summative Assignment 2: Simulation Technical Implementation · Unity 6 (6000.6) · URP · Android

GitHub repository	https://github.com/Strixx-thierry/AR-Pre-Flight-Drone-Inspection-Trainer
Mobile build	Android APK (ARM64, IL2CPP, min API 26): [build link]
Video demonstration	[video link]

1. Overview

The application trains drone pilots to perform the four-point pre-flight inspection defined in the Assignment 1 SRS: battery latch security, the condition of all four propellers, gimbal-clamp removal and controller calibration. The trainee picks an aircraft (SKYFOLD F4 or TALON X4), chooses **Guided** or **Assessment** mode, and inspects a full-scale 3D drone. Every run randomises its faults (cracked blades, unlatched or low battery, fitted clamp), so the trainee must actually look at the parts. Unsafe launches are blocked and logged, and each session ends with a scored report that is saved locally.

Flow: Splash → Home → Hangar (aircraft select) → Briefing (mode) → Inspection → Launch gate → Launch → Report → Records. Pause, Settings, Help and a tracking-loss overlay can appear at any point.

2. Final architecture

The project is split into two assemblies. `DroneTrainer.Runtime` holds all game code. `DroneTrainer.Editor` holds the build tooling, which generates the scenes, prefabs, theme and audio from code. Runtime code is organised into subsystems with one responsibility each. They communicate through C# events (Section 5), so input flows from the raycaster, through the task logic, to the UI.

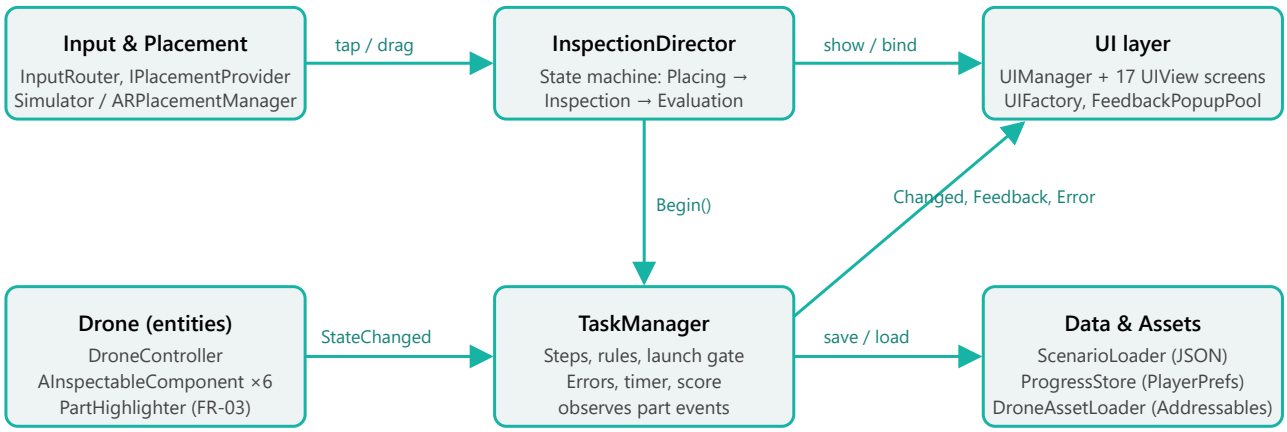


Figure 1: Runtime subsystems and the event flow (input → logic → presentation).

Subsystem	Key classes	Responsibility
Placement / view	`IPlacementProvider`, `SimulatorPlacement`, `ARPlacementManager`	Puts the drone in the scene and drives the camera (orbit, pan, zoom, focus on a part).
Drone entities	`DroneController`, `AInspectableComponent` + subclasses	Part state, picking by raycast, ghost preview, prop spin.
Simulation rules	`TaskManager`, `ScoreCalculator`	Step order, validation, launch gate (FR-08), error log, timer (FR-09), score.
Flow control	`MenuDirector`, `InspectionDirector`, `StateMachine`	Screen flow and the high-level simulation states.

Presentation	`UIManager`, `UIView` + 17 views, `UIFactory`, `UITheme`	HUD, panels and modals from one design system.
Data, audio, assets	`ScenarioLoader`, `ProgressStore`, `AudioManager`, `DroneAssetLoader`	Configuration, persistence, sound and runtime asset loading.

3. Simulation design

Entity	Interaction	States and rule
Battery latch (`BatteryLatchModule`)	Tap, then slide-to-lock; Swap button	Locked/Unlocked, charge 1–4 LEDs. Safe only when locked *and* charge ≥ 2 (a low pack must be swapped).
4 × propeller (`RotorModule`)	Tap, inspect (zoom/rotate), then an *Airworthy/Damaged* verdict; Replace	Unchecked → Passed / Faulty → Fixed. Passing a damaged blade is a safety error; a false alarm is a minor error.
Gimbal clamp (`GimbalCoverModule`)	Swipe up (on the part or the panel)	Fitted → Removed, with a slide-off animation.
Controller (`CalibrationView`)	Two virtual sticks: centre → 8-point circles → centre	Finishing early is logged as "incomplete calibration".

Cause and effect. In Guided mode the parts of the current step glow (rim-light outline shader) and later steps are locked. *Launch Check* refuses the launch while any critical item is unsafe: it adds a safety error, opens the "Launch blocked" modal and points to the failing part. When everything passes, the motors spin up (audio and animation), the drone lifts off, and the report shows the time, errors, per-step breakdown and a score (100 − 10 per safety error − 2 per minor error − a time penalty; a pass needs ≥ 70 and all steps done).

Fault variation. Each scenario JSON holds a `faultPool` (0–2 cracked rotors, 80% unlatched battery, 35–50% low charge, 85% fitted clamp). The weighting follows the UK CAA's CAP 2534, which identifies pre-flight checking and battery/power management as the leading human-error contributors to drone equipment failures. A damaged blade is modelled with a snapped tip, chipped edge and stress-whitening, so it can be spotted only by inspecting closely.

4. OOP implementation

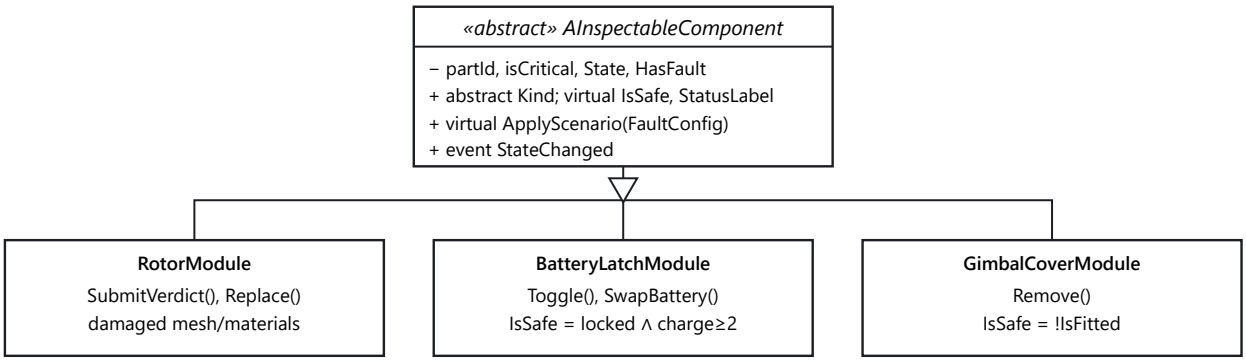


Figure 2: Inspectable-part hierarchy (inheritance and polymorphism).

- **Encapsulation:** each part keeps its flags private (`IsLocked`, `HasFault`, `IsFitted` have private setters). Other code changes a part only through safe methods (`Toggle()`, `SubmitVerdict()`, `Remove()`). `TaskManager` exposes read-only state and actions; its arrays and the error list are internal.
- **Abstraction:** `AInspectableComponent` defines what an inspectable part is (id, criticality, state, safety, scenario reset) without implementation details. `IPlacementProvider` abstracts "where the drone lives and how the camera moves", so the director does not know whether AR or the 3D viewer is active.
- **Inheritance:** three concrete modules extend the base, and seventeen screens extend `UIView`, which provides the build/bind/show lifecycle. Inheritance is used only where behaviour is genuinely shared.
- **Polymorphism:** `DroneController` and `TaskManager` iterate over a list of `AInspectableComponent` and call `IsSafe`, `ApplyScenario()` and `StatusLabel` without type checks. Each override applies its own rule (for example the battery's charge condition). The director pattern-matches on the type only to choose which panel to open.

5. Design patterns and event system

Pattern	Where	Problem it solves
Observer (C# events)	Parts raise `StateChanged`; `TaskManager` raises `Changed`, `Feedback`, `ErrorLogged`, `LaunchBlocked`; views raise button events	Parts don't know about tasks, and tasks don't know about the UI. The HUD, audio and pop-ups subscribe independently, so new reactions need no changes to the simulation logic.
State	`IState`, `StateMachine`, and `PlacingState` / `InspectionState` / `EvaluationState` in `InspectionDirector`	Each phase has its own enter, tick and exit behaviour (timer, input routing, launch sequence) without one large switch statement.
Factory	`UIFactory`, `ChecklistRow.Create()`	Checklist rows, buttons and panels are built from JSON steps and theme tokens. The same factory serves the editor scene builder and runtime lists.
Object Pool	`FeedbackPopupPool` (`UnityEngine.Pool.ObjectPool`)	Every action spawns a floating label. Pooling avoids an Instantiate/Destroy per event (see below).
Singleton	`AudioManager` (lazy, `DontDestroyOnLoad`)	One audio source that survives scene changes and is reachable from any view.

Object pool implementation

`FeedbackPopupPool` creates an `ObjectPool<FeedbackPopup>` with a default capacity of 6 and a maximum of 12, and pre-warms 6 instances on start-up. `Spawn()` calls `Get()` (which activates the object), projects the

part's world position into HUD space and calls `Play()`, passing `pool.Release` as the completion callback. When the rise-and-fade animation ends, the popup releases itself (`actionOnRelease` deactivates it), ready to be reused. Before reuse, `Play()` resets text, colour, position, scale and alpha. This removes garbage-collection spikes and canvas rebuilds during the inspection, which matters at the 30 fps mobile target.

6. Data persistence (DMP)

- **Configuration:** scenario steps, instructions, hints and fault pools are JSON files in `StreamingAssets/Scenarios`, read with `UnityWebRequest` on Android. Changing a checklist needs no recompile (NFR maintainability). A bundled copy acts as a fallback.
- **Runtime state:** the timer, current step, error list and active faults live in memory in `TaskManager`.
- **Persistent sessions:** `ProgressStore` serialises a `SessionResult` (drone, mode, time, score, pass, safety/minor errors, per-step times, error log, timestamp) to JSON in `PlayerPrefs`, capped at 20 sessions. The Records screen loads them on launch and shows the most recent 8 with time, score and pass/fail, plus best time, pass rate and average errors. Settings (sound, haptics, hints) are also stored in PlayerPrefs.

7. Audio architecture

`AudioManager` plays one-shot SFX by name from an `AudioLibrary` asset and runs a separate looping source for the motors. The library maps 13 named clips (*tap, select, success, error, warning, confirm, latch, swipe, tick, found, place, blocked, motor*). The clips were synthesised by the editor tooling, and a designer can replace any of them by dropping in a file with the same name. Sounds are tied to meaning, not decoration: latch click and haptic on locking, an error buzz on a missed fault, an alarm when a launch is blocked, ticks as calibration points are reached, and a motor spin-up whose pitch rises with RPM during launch. Sound and haptics can be switched off in Settings.

8. Asset management (Addressables)

The two 3D drone models (meshes, materials, damaged-propeller variants) are the heaviest runtime assets. They are registered in an Addressables group **"Drones"** with the label `drone` and the address equal to the drone id (`SKYFOLD_F4`, `TALON_X4`). `DroneDefinition` stores only the address string, not a hard prefab reference, so the menu catalog in `Resources` never pulls the models into memory. When a session starts, `DroneAssetLoader.Load()` calls `Addressables.LoadAssetAsync<GameObject>(address)`, waits for the handle and instantiates the result. When the inspection scene is left, `InspectionDirector.OnDestroy()` disposes the loader, which destroys the instance and calls `Addressables.Release(handle)`, so only the selected drone is ever resident. The menus use lightweight 2D renders of the drones instead of the 3D models.

9. Performance on mobile

- URP mobile renderer at 0.8 render scale, a single directional light with soft shadows, and no post-processing in the inspection scene.
- Highlight outlines are a lightweight unlit additive shader drawn on shell meshes, tinted with `MaterialPropertyBlock` (no material instances).
- Models are about 20k triangles per drone. UI art is pre-rendered as 9-sliced sprites, and SFX are decompressed on load to avoid playback cost.
- Pooled feedback pop-ups and on-demand Addressables loading keep allocations and memory low during a session.

10. Deviations from the original SRS

SRS	Implemented	Justification

AR plane detection and placement on a real surface (FR-01/02)	A 3D inspection viewer: orbit including underneath, pan, zoom, and automatic focus on the inspected part. `ARPlacementManager` (plane detection, raycast, anchoring, tracking monitor) remains in the code behind `GameSession.UseAR`.	Agreed with the lecturer: close-up inspection of small parts such as blade tips is more reliable and usable with free camera control than by moving a phone around a desk. It also removes the dependence on lighting and ARCore support.
One drone	Two drones (SKYFOLD F4, TALON X4) with separate scenarios	Demonstrates the data-driven design: same modules, a different JSON per aircraft.
Fixed scenario	Randomised faults per run, including low battery	Prevents memorisation and reflects real failure causes (CAA CAP 2534).
FR-08 wording ("allows the user to proceed…")	Launch is *prevented* while critical items are unsafe	Corrects the intent of the original requirement.

11. Limitations and future improvements

- The AR mode exists but was not enabled for the final build. A future version would let users switch between AR and 3D in Settings.
- Persistence uses PlayerPrefs (local, single user). A file- or cloud-based store would support multiple trainees and instructor reports.
- Fault types are limited to four systems. Motor bearing noise, arm locks, compass calibration and GPS lock are natural extensions, as is a post-inspection flight test.
- Drone models are procedural, not manufacturer CAD, so they are representative rather than exact.

Reference: UK Civil Aviation Authority, CAP 2534 (drone occurrence analysis) — caa.co.uk/our-work/publications/documents/content/cap2534